

DYNAMIC ETA PREDICTOR

Detailed technical design, data-science lifecycle, robustness controls, and mountain-region deployment plan

PROJECT FOCUS	Real-time ETA quantiles for long and mountainous routes, built around live routing traffic, terrain, weather, geospatial context, and completed-trip learning.
CURRENT MODE	Safe development serving. P50 is anchored to the live traffic-routing estimate until a chronological held-out evaluation certifies a learned model.
REPORT DATE	2026-09-29

Purpose

This report documents what the repository does today, why a mountain ETA product must be built as a routing-baseline correction, the precise data contracts and model behavior, and the criteria required before a learned model can affect live ETAs. It distinguishes implemented behavior from recommended future production work.

1. Executive summary
2. Problem framing and design principles
3. System architecture
4. Live data ingestion
5. Trip event collection and storage
6. Feature engineering for mountainous routes
7. Model architecture and quantile semantics
8. Training objective and data contract
9. Chronological evaluation and promotion
10. Metrics and interpretation
11. Serving behavior and safeguards
12. Testing and quality controls
13. Operations, observability, and security
14. Known limits and data requirements
15. Practical runbook
16. Implementation inventory and glossary

Reading note

The report uses the term **baseline** for the traffic-aware routing ETA. The model predicts ordered ETA multipliers around that baseline. In a future mature system, the same concept can be expressed as an additive residual in minutes; the critical engineering principle is that the learned component corrects a routing engine rather than replacing route computation.

1 Executive summary

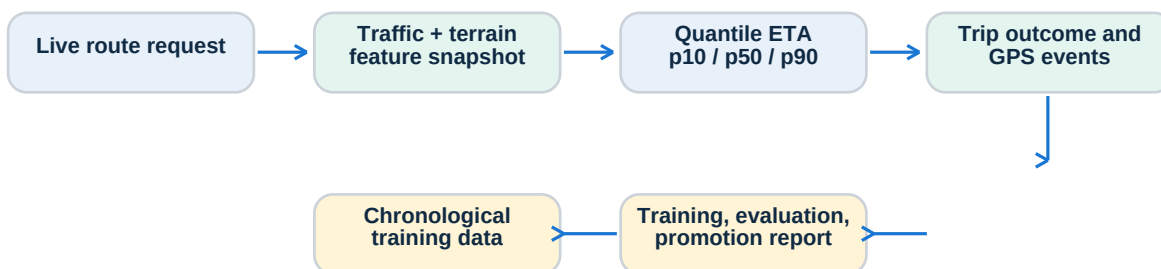
The project is a FastAPI service and PyTorch learning pipeline for predicting route-level travel-time uncertainty. A request provides an origin, destination, and optional vehicle or driver behavior inputs. The service obtains a route and a traffic-aware duration from TomTom when a key is available; an OSRM route is retained only as a clearly labeled static development fallback. It samples the route, enriches it with elevation, inferred grade, weather, and H3 geospatial cells, and passes these values into a compact neural quantile model.

The service returns three times in minutes: p10, p50, and p90. P50 is the central ETA. P10 is an optimistic bound and p90 is a conservative bound. The architecture guarantees strict ordering, so it cannot output p90 below p50 or p50 below p10.

The most important safety decision is intentional: the bundled model checkpoint has not been trained on real completed mountain trips. Therefore live serving does not let it convert a credible ten-hour live routing estimate into an implausible two-hour prediction. In default development mode the traffic ETA is the p50 anchor and the interval is conservative. Learned values are activated only when a candidate checkpoint has a matching, eligible chronological evaluation report.

This is a strong foundation for a reliable ETA product, but not a claim that the system is already fully industry-grade. Real accuracy needs representative labels: actual trip completions across corridors, seasons, elevation bands, drivers, vehicles, and disruptive conditions. Operationally, a production deployment also needs authenticated event intake, managed storage, monitoring, model registry controls, alerting, privacy policy, and incident response.

Core lifecycle



Production principle: a live routing ETA is the baseline; learning is a measured correction, not a substitute for road routing.

2 Problem framing and design principles

Long mountain journeys expose the limits of a distance-only ETA. A road network may report the correct legal route while missing transient effects such as long climbing sections, switchbacks, altitude, weather, temporary control points, low visibility, convoying, road damage, variable vehicle capability, and seasonal traffic behavior. The error is often asymmetric: an optimistic ETA can cause a passenger, dispatcher, or rescue workflow to make an unsafe plan.

The design follows the DeeprETA idea from the supplied reference paper: begin with a routing-engine ETA and learn the residual behavior from historical outcomes. The paper's broader lessons are used here, not copied mechanically: feature interactions matter; location needs compact learned representations; regular chronological retraining and validation are essential; and routing estimates remain valuable inputs.

Principle	Implementation in this project	Why it matters
Routing is primary	TomTom traffic-aware route duration is the live baseline. OSRM is static fallback only.	The neural model does not need to rediscover the road graph or live traffic from sparse features.
Learning is conditional	The neural network receives route features plus driver inputs and predicts three ordered multipliers.	The same distance can behave differently under elevation, grade, weather, and driver conditions.
Uncertainty is first-class	P10, p50, p90 are trained with a quantile loss and checked for empirical coverage.	An ETA must communicate risk, not only a single convenient number.
Future data stays unseen	Older records train, newer records validate, newest records test.	Time-aware splitting limits leakage from changing traffic, weather, and road conditions.
Promotion is gated	A report checks MAE, baseline skill, coverage, and quantile order before learned serving can be enabled.	A saved checkpoint is not evidence that it is safe to use.

A model that is accurate on shuffled historical rows can fail badly when conditions shift. For ETA work, temporal realism is more valuable than an impressive random-split metric.

3 System architecture

The repository is organized around four responsibilities: API serving, live data providers, durable trip-event collection, and offline model training. This separation is deliberate. Serving needs bounded upstream calls and a safe fallback. Training needs immutable snapshots of what was known when the ETA was requested. Storage needs to retain the target label only after the trip completes.

Layer	Key module	Responsibility
API	src/api/app.py	FastAPI endpoints, model loading, route resolution, feature construction, p10/p50/p90 response, trip lifecycle endpoints, and health reporting.
Live traffic	src/data_collection/realtime_provider.py	TomTom geocoding, traffic-aware route calculation, and traffic-flow observations through a bounded adapter.
Feature assembly	RouteFeatureProvider in app.py	Route sampling, elevation lookup, weather lookup, grade calculation, and H3 conversion.
Event store	src/data_collection/trip_store.py	SQLite trip snapshots, GPS events, completion labels, and export to the offline JSONL contract.
Model	src/models/deepreta_system.py	Route encoder, hashed H3 embeddings, discretized physical features, low-rank sequence attention, and ordered quantile head.
Training	src/training/train.py	Strict data validation, chronological split, pinball-loss training, test evaluation, promotion report, and checkpoint save.
Configuration	config/project_config.yaml	Feature order, dimensions, split fractions, minimum sample sizes, and promotion thresholds.

The application has a model-free safety path. If the model cannot load, the API returns a service error rather than inventing a model output. If the model loads but has no verified promotion report, it still executes the feature/model path for collection and diagnostics but serves the traffic-baseline anchor rather than an unchecked learned adjustment.

Important boundary

SQLite is an appropriate durable local development store, not the final multi-instance event platform. The schema and endpoint contract are designed so that a managed relational database and event stream can replace it without changing the prediction payload.

4 Live data ingestion

At prediction time the service resolves an origin and destination into a route. With a configured TomTom key, it geocodes both places, asks for the fastest traffic-aware driving route, and reads route geometry, distance, traffic travel time, free-flow travel time, and traffic delay. It also attempts to obtain a Traffic Flow Segment observation at the route origin. The route provider is isolated behind a typed adapter so that its response format does not leak through the rest of the product.

Input	Source	Used for	Failure behavior
Origin and destination	API request / TomTom geocoder	Route selection and response labeling.	Invalid or unavailable geocoding returns an explicit request/upstream error.
Route geometry and traffic duration	TomTom Routing API	Baseline ETA and points for feature extraction.	Without a live key, a static OSRM fallback is labeled as no live traffic.
Road-segment speed	TomTom Traffic Flow	Response diagnostics and future traffic features.	Optional; prediction continues if a flow observation is unavailable.
Elevation	Open-Meteo elevation endpoint	Terrain profile and grade estimation.	Falls back to zeros; this should be monitored because it weakens mountain signal.
Weather	Open-Meteo forecast endpoint	Temperature, humidity, wind, precipitation.	Uses documented neutral defaults when temporarily unavailable.

The API client uses a timeout and simple in-memory caching. Nominatim calls used by the OSRM development fallback are rate-limited. These are responsible local-service practices, but high-volume production should use provider-specific quotas, retry budgets, circuit breakers, cache observability, and a managed geocoding/routing agreement.

The current architecture intentionally keeps all upstream data acquisition synchronous and bounded. This makes local behavior understandable. For high QPS deployment, cache warmup, asynchronous provider adapters, per-provider health checks, and a background feature service would reduce tail latency.

5 Trip event collection and label creation

A learning system cannot train from predictions alone. It needs the eventual outcome. The API therefore supports a trip lifecycle: create a trip at dispatch, submit position events while it is active, complete it with an actual duration, and optionally request a remaining ETA from the latest position. The initial feature snapshot is stored at trip creation so training compares the actual duration against exactly the conditions and routing ETA known at the time of prediction.

```
POST /api/trips -> stores prediction + feature snapshot and returns trip_id
POST /api/trips/{trip_id}/positions -> stores timestamped GPS ping
POST /api/trips/{trip_id}/eta -> recomputes remaining ETA from latest GPS point
POST /api/trips/{trip_id}/complete -> stores actual trip duration label
python -m src.training.export_events --output data/training/mountain_routes.jsonl
```

The local store has a trips table and a position_events table. Position events are keyed by trip and timestamp, making repeated delivery idempotent. The trip record contains request JSON, the prediction response, the full feature snapshot, completion timestamp, actual duration, and source metadata. Write-ahead logging is enabled for more resilient local SQLite behavior.

Field	Meaning	Data-science role
trip_id	Stable unique trip identifier.	Deduplication key. The trainer rejects duplicates.
created_at / started_at	Time when the request snapshot was created.	Chronological split key, because it is when features were available.
feature_snapshot	H3 cells, N x 6 features, driver profile, routing ETA, request metadata.	Feature vector that must match what production served.
actual_duration_minutes	Observed completed duration.	Supervised training target.
position_events	Optional live GPS pings with speed and heading.	Supports remaining-ETA refresh and future sequence features.

Data quality warning

Manual completion labels are useful for a prototype but need audit controls in deployment. Production ingestion should validate device clocks, detect impossible motion, protect against duplicate trips, attach a source/device identity, and preserve raw events separately from derived labels.

6 Feature engineering for mountainous routes

The feature provider samples a fixed number of points along the live route. The configured sequence length is eight points. Sampling keeps inference compact, but it also means each point must represent a substantial section of a long route. At every sampled location, the pipeline derives an H3 cell and a six-column physical feature row.

Feature	Definition	Mountain relevance	Current range handling
Elevation (m)	Height above sea level at the sampled point.	Altitude correlates with vehicle performance, weather exposure, road type, and closures.	Bucketed using fixed physical bounds of -500 to 9,000 m.
Grade (%)	Elevation change divided by approximate horizontal run between route points.	Climbs, descents, braking, and switchbacks cause duration changes beyond distance.	Clamped to -30% to 30% before bucketization.
Temperature (C)	Current forecast temperature near each sample.	Heat, frost, snow, and visibility-related conditions influence pace.	Bucketed using -50 to 60 C bounds.
Humidity (%)	Current relative humidity.	A useful proxy for wet or fog-prone conditions, but not a direct road-surface reading.	Bucketed from 0 to 100%.
Wind (km/h)	Current 10 m wind speed.	Can affect high passes, exposed roads, and vehicle stability.	Bucketed from 0 to 200 km/h.
Precipitation (mm)	Current precipitation measurement.	Directly relevant to visibility, traction, and disruptions.	Bucketed from 0 to 200 mm/h.

Geospatial features use H3 cells at resolution 7. Rather than retaining a massive geographic vocabulary, cells are deterministically hashed into a fixed embedding table. This is memory-efficient and allows unseen cells to map somewhere, although collisions mean it is not as expressive as a purpose-built geographic registry. The supplied DeepETA paper uses multi-resolution geospatial feature interactions; a mature version should add origin, destination, origin-destination pair, and multi-resolution cell tokens.

Current limitations of feature construction are important: weather is point-in-time rather than forecast along expected arrival time; traffic flow is observed only near the route origin; grade is inferred from sparsely sampled elevation; road curvature, surface quality, closures, snowfall, restrictions, vehicle payload, and historical corridor speed profiles are not yet model inputs.

7 Model architecture and quantile semantics

The core PyTorch model is **DeeprETAEndToEndSystem**. It is intentionally compact. It encodes a variable-length route, fuses a three-element driver profile, and outputs three ETA multipliers. The multiplier representation keeps the learned function focused on how actual travel differs from a routing engine's base duration.

Stage	Mechanism	Output
Spatial encoding	MD5-based fixed-vocabulary index into an H3 embedding table.	A learned vector per sampled geographic cell.
Continuous encoding	Each physical feature is discretized into fixed physical-range buckets and embedded.	A learned vector for elevation, grade, weather state, and precipitation at each point.
Route token	Concatenate spatial and continuous embeddings, then linearly project.	One dense token per route sample.
Sequence interaction	Low-rank projection of key/value route tokens with attention-style context.	Route-aware token representations at low memory cost.
Driver fusion	Three driver indicators pass through a small MLP and are fused into every token.	Driver-conditioned route representation.
Quantile head	Average pool then MLP and three constrained output values.	Strictly ordered p10, p50, p90 multipliers.

Ordered quantiles are guaranteed algebraically. The head forms a positive median with softplus. P10 is the median multiplied by a sigmoid fraction, which makes it smaller than the median. P90 is the median plus a positive softplus gap. This is better than predicting three unconstrained values and sorting them after the fact, because the model is trained in a quantile-consistent parameterization.

```
median = softplus(raw_median) + epsilon
p10 = median * sigmoid(raw_lower)
p90 = median + softplus(raw_upper) + epsilon

final ETA quantile in minutes = routing baseline minutes * predicted multiplier
```

The name 'Linformer' in earlier discussion should be treated carefully. The implementation uses low-rank sequence projections to keep attention small. It is not a literal reproduction of every component from the DeeprETA reference architecture. The small model choice is reasonable for latency and cost, but model accuracy must be demonstrated through held-out data rather than assumed from architecture.

8 Training objective and data contract

The trainer accepts JSON Lines rather than a loose CSV. This forces a single route-level example to contain its route sequence, driver inputs, baseline duration, actual duration, identity, and observation time. It rejects raw telemetry CSV files because telemetry without reconstructed route features and an actual trip label is not a directly trainable ETA example.

```
{"trip_id": "trip-123", "started_at": "2026-09-29T09:00:00+00:00",  
  "h3_cells": ["876...", "876..."],  
  "continuous_features": [[210, 0.2, 29, 65, 8, 0], [220, 0.4, 29, 64, 9, 0]],  
  "driver_profile": [1.0, 0.2, 0.3],  
  "base_duration_minutes": 120, "actual_duration_minutes": 135}
```

The target is a multiplier: actual duration divided by baseline duration. In the example above, the target multiplier is 1.125. A p50 multiplier near 1.125 turns a 120-minute traffic baseline into a 135-minute median ETA. If the baseline changes at inference time, the same learned correction scales with the current routing estimate.

The loss is the pinball, or quantile, loss. For quantile q , it penalizes underprediction and overprediction asymmetrically so that the optimum estimates the q -th conditional quantile. The trainer averages loss over $q = 0.10, 0.50$, and 0.90 . P50 is not trained as a conventional mean-squared regression target; it is trained as a median estimate, which is more robust to a few unusually delayed mountain trips.

```
error = actual_multiplier - predicted_multiplier  
pinball(q) = max((q - 1) * error, q * error)  
training loss = mean(pinball(0.10), pinball(0.50), pinball(0.90))
```

Validation discipline is embedded in the trainer. Input rows must have finite numeric values, matching $N \times 6$ feature arrays, exactly three driver values, positive durations no longer than seven days, a non-future timezone-aware feature time, and no duplicate trip IDs. These checks stop common silent failures before GPU or CPU training begins.

9 Chronological evaluation and checkpoint promotion

Randomly shuffling route records makes validation optimistic when traffic, weather, route access, mapping data, and operations change over time. The project now sorts completed records by **started_at**, the time that the model's snapshot was available. This matters more than sorting by completion time: features cannot use knowledge that became available after dispatch.

Partition	Default allocation	Allowed use
Training	Oldest period; at least 100 rows.	Optimize neural-network parameters.
Validation	Next 15% with at least 20 rows.	Choose the best epoch by validation pinball loss.
Test	Newest 15% with at least 20 rows.	One final quality measurement. Never select model weights with it.

The current fractions and count limits require at least 144 completed trips before training is allowed. This is a mechanical minimum, not a claim of statistical sufficiency. For reliable coverage claims across many mountain conditions, the real requirement is likely thousands of representative trips. The trainer also keeps equal observation timestamps within one partition, preventing a batch of simultaneously produced records from being split across train and test.

After fitting, the best validation epoch is restored. The test metrics are computed once and saved with the candidate checkpoint. The adjacent report includes a SHA-256 checksum of the checkpoint. Serving compares the report's checksum to the configured model file, preventing accidental pairing of a good report with a different checkpoint.

Promotion gates are deliberately conservative. A candidate must have no observed quantile crossings, p50 MAE below the configured limit, non-negative p50 skill relative to the live routing baseline, and p10/p90 coverage within configured acceptance ranges. Failures are recorded in the report. A candidate may still be retained for investigation, but it cannot enable learned live quantiles.

10 How to evaluate whether the model is good

A good ETA model is not defined by low training loss. It is defined by future, unseen trip outcomes. For every test trip i , retain the baseline B_i , actual completion A_i , and model quantiles $Q10_i$, $Q50_i$, $Q90_i$ in minutes. Evaluate the same set against both the model and the baseline.

Metric	Formula or interpretation	Desired behavior
P50 MAE	$\text{mean}(\text{abs}(Q50 - A))$ in minutes.	Lower than TomTom baseline MAE; report median absolute error too for robustness.
P50 skill	$(\text{baseline MAE} - \text{model MAE}) / \text{baseline MAE}$.	Positive. For adoption, seek a practically meaningful margin, not merely 0.1%.
P10 coverage	fraction of $A \leq Q10$.	Near 10%. Too high means p10 is overly pessimistic; too low means it is dangerously optimistic.
P50 coverage	fraction of $A \leq Q50$.	Near 50%. A persistent offset reveals median bias.
P90 coverage	fraction of $A \leq Q90$.	Near 90%. Too low means the confidence bound fails too often.
Interval width	$Q90 - Q10$.	As narrow as possible while still achieving coverage; narrow intervals alone are not success.
Pinball loss	Proper scoring rule evaluated separately at each quantile.	Lower is better; compare model versus a baseline quantile policy.

Evaluation must be sliced. Report each metric by route/corridor, distance band, elevation band, steepness, day/night, precipitation state, season, vehicle type, driver class, and traffic delay. A model that improves city-adjacent roads but fails high passes is not a good mountain ETA model even if aggregate MAE is lower.

For a mature evaluation program, add confidence intervals by bootstrapping trips or by time blocks, compare against simple historical-delay and gradient-boosting baselines, and run a shadow deployment before making the learned output customer-visible. Calibrate quantiles on a validation period only; never tune calibration against the final test period.

11 Serving behavior and safety safeguards

At startup the API loads the configured PyTorch checkpoint. It then decides whether learned quantiles are permitted. The environment must explicitly request calibrated mode with `ETA_MODEL_CALIBRATED=true`. Even then, the service reads the adjacent metrics report, verifies report schema version, checks `promotion.eligible`, and compares the checkpoint SHA-256. Any missing, failing, or mismatched condition leaves learned mode disabled.

Serving state	Returned p50	Returned interval	Why
Default development / uncalibrated	Traffic-aware routing ETA.	80% to 130% of baseline.	Prevents an arbitrary bundled checkpoint from reporting a much shorter mountain ETA.
Eligible calibrated candidate	Model p50 multiplier x live baseline.	Model p10/p50/p90 multiplier x live baseline.	Enabled only after the chronological report passed the configured gates.
Model unavailable	No prediction.	HTTP 503 with health diagnostic.	Avoids returning fabricated values when model loading is broken.
No TomTom key	Static OSRM route ETA.	Clearly labeled no-live-traffic path.	Useful for local development, not equivalent to real-time traffic serving.

The response contains the source provider, traffic baseline, free-flow duration, traffic delay, traffic-flow observation when available, feature-point count, and model-status text. This makes it possible for clients and dashboards to distinguish a true live-traffic estimate from development fallback behavior.

The model is still invoked in default serving mode. This validates the live feature path and preserves a route-level snapshot for trip collection, but its learned output is intentionally not allowed to override the safe baseline until qualification. This is a practical compromise between feature-pipeline verification and user safety.

12 Testing and quality controls

The repository has unit and smoke tests in `tests/test_quality.py` plus a standalone inference check. The tests are designed to cover contract failures that are easy to miss in a model project: output ordering, variable route lengths, configuration/model compatibility, rejection of raw CSV input, live feature handoff to the model, mocked TomTom response parsing, trip export contract, chronological split ordering, promotion rejection, and calibration-report checksum matching.

```
python -m compileall -q src tests test_inference.py
python -m unittest discover -s tests -v
python test_inference.py
```

Control	What it catches	What it does not prove
Ordered quantile test	p10 < p50 < p90 for supported route lengths.	That intervals are empirically calibrated.
Chronological split test	Newest records do not leak into train rows.	That the data is representative of future mountain conditions.
Promotion test	A worse or uncalibrated candidate is rejected.	That thresholds are correctly chosen for all business use cases.
TomTom adapter mock	Expected response fields are translated into internal types.	Provider behavior under quota, latency, schema changes, and outages.
Trip export test	Stored snapshot becomes a trainer-valid record.	GPS integrity or correctness of manually entered actual duration.

Before production, add integration tests against a sandbox/provider test account, schema-contract tests for external payloads, deterministic training smoke tests with a small fixture dataset, load tests for upstream latency, property-based tests for malformed events, and tests for concurrent trip updates. Version both data schema and model/feature code so an older snapshot can be reproduced exactly.

13 Operations, observability, privacy, and security

Machine-learning correctness is only one part of an ETA service. Production operation requires visibility into upstream health, data quality, model behavior, and user impact. The API's health endpoint currently reports routing-provider mode, model-load status, and whether learned quantiles are enabled. This should become a monitored service-level signal rather than a manual inspection tool.

Area	Recommended production control	Reason
Secrets	Use a managed secret store and rotation policy for routing keys. Never commit or paste production keys into source files.	Traffic access is billable, rate-limited, and security-sensitive.
Identity	Authenticate trip creation, position pings, completion, and admin/model actions.	Unprotected endpoints allow label poisoning and location-data leakage.
Location privacy	Define retention, access, deletion, minimization, and consent policy for GPS traces.	Trip location history is sensitive personal and operational data.
Observability	Emit structured metrics for latency, provider errors, cache hit rate, fallback rate, missing elevation/weather, model version, and ETA distributions.	Without this, a silent fallback or provider regression can degrade ETAs unnoticed.
Data quality	Alert on missing completions, impossible speeds, unusual target ratios, duplicate devices, and feature drift.	A model can degrade because its labels or inputs degrade.
Release control	Use immutable artifact storage, signed/model-registry metadata, canary and shadow releases, and rollback.	A checksum-bound local report is helpful but not a complete enterprise model registry.

For production model governance, persist the training-data window, git revision, configuration, feature schema version, metric report, approval identity, and candidate artifact hash together in an immutable registry. The current report is a good local artifact, but manual report editing is still possible without a signing workflow. A managed registry or an HMAC/signature key is required when promotion authority must be independently verified.

14 Known limitations and data requirements

The system is deliberately honest about current evidence. There is no complete historical mountain-trip corpus in the repository. The bundled checkpoint is therefore not evidence of model accuracy. The current safe baseline mode is the correct behavior until data arrives.

Gap	Risk if ignored	Recommended next addition
Insufficient completed trips	A model may learn noise or a few routes, then generalize badly.	Collect thousands of completed trips across diverse routes and seasons.
Sparse route sampling	Eight points can miss short severe segments, closures, and switchback density.	Use adaptive sampling by route length, grade change, curvature, and risk segments.
Point-in-time weather	Conditions may change before arrival at a distant pass.	Join forecast weather by projected arrival time and route segment.
Limited traffic feature	One origin flow segment cannot represent an entire long route.	Collect flow/incidents along the route and aggregate corridor features.
No map restrictions/closures	Model cannot know a route is closed or convoy-controlled.	Ingest road closures, incident feeds, local authority advisories, and access restrictions.
No vehicle load or road condition	Similar routes may have different physical travel constraints.	Add vehicle class, weight/load, drivetrain, tire/chain condition, and verified road state where lawful.
Local SQLite	Contention, availability, and retention limits for multi-user deployment.	Move to managed Postgres plus durable event stream/object storage.

The most valuable data is not more generic GPS traces. It is accurately linked dispatch-time feature snapshots, routing baseline, periodic route state, and verified actual arrival time. Labels should cover difficult examples deliberately: high passes, monsoon conditions, snow/ice where applicable, roads with restrictions, night driving, tourist surges, and long-duration itineraries.

15 Practical operating runbook

1. Start safely. Configure a rotated routing key outside source control. Run the service with the default uncalibrated behavior. Confirm `/health` reports the expected provider and `learned_quantiles_enabled=false`.

```
source env/bin/activate
export TOMTOM_API_KEY='stored-outside-source-control'
uvicorn src.api.app:app --reload
```

2. Collect labels. Create every actual trip through `POST /api/trips`, send periodic GPS positions, and complete the trip with a verified duration. Avoid mixing hand-entered test trips with operational data unless they are clearly tagged and excluded from production training.

3. Inspect data. Check counts by route, month, elevation, weather, and completion source. Investigate extreme actual/baseline ratios before training; they may be legitimate disruptions or bad labels.

4. Export and train a candidate. Do not overwrite the bundled checkpoint. Save candidates under a dated or versioned models directory. The training command produces both a checkpoint and an adjacent metrics report.

```
python -m src.training.export_events --output data/training/mountain_routes.jsonl
python -m src.training.train --data data/training/mountain_routes.jsonl --output
models/mountain_eta_v001.pth
python -m json.tool models/mountain_eta_v001.metrics.json
```

5. Review metrics. Compare p50 MAE and pinball loss to TomTom and simple non-neural baselines. Inspect p10/p50/p90 coverage overall and in critical mountain slices. Review promotion failures; do not simply relax thresholds to force eligibility.

6. Shadow deploy first. Load the candidate in an internal/shadow environment and log its output alongside the safe live baseline. Compare outcomes for a new future period. Only then enable `ETA_MODEL_CALIBRATED=true` with the checkpoint and its matching report.

```
ETA_MODEL_PATH=models/mountain_eta_v001.pth \
+ETA_MODEL_CALIBRATED=true \
+uvicorn src.api.app:app
```

7. Monitor and rollback. If live p90 undercoverage, provider fallback rate, or segment MAE worsens, disable learned quantiles immediately by removing `ETA_MODEL_CALIBRATED=true` and investigate using preserved snapshots and model metadata.

Implementation inventory

Artifact	Role in current project
README.md	Local setup, API usage, trip collection examples, training command, and calibration guard explanation.
config/project_config.yaml	Feature and model dimensions plus validation/test/promotion settings.
src/api/app.py	Serving, live route features, safe p50 anchor, trip endpoints, and calibration-report gate.
src/data_collection/realtime_provider.py	Typed TomTom live route and flow adapter.
src/data_collection/trip_store.py	SQLite event persistence and chronological JSONL exporter.
src/models/deepreta_system.py	Compact sequence encoder and mathematically ordered quantile head.
src/training/train.py	Data validation, chronological split, training, metrics, report, and promotion decision.
tests/test_quality.py	Model, ingestion, chronology, promotion, provider, and API safety checks.

Glossary

Term	Meaning
Baseline ETA	Live traffic-aware route time from a routing provider before learned correction.
P10 / P50 / P90	Conditional ETA quantiles. Approximately 10%, 50%, and 90% of matched outcomes should be at or below each value.
Pinball loss	Quantile-regression objective that rewards calibrated percentile estimates.
Coverage	Observed fraction of actual durations at or below a predicted quantile.
H3 cell	Hierarchical hexagonal geospatial index used as a compact location token.
Feature snapshot	The immutable inputs available when an ETA was predicted, stored for later supervised learning.
Chronological split	Training/evaluation separation by time rather than random shuffle.
Promotion gate	A required set of held-out metrics that must pass before learned predictions can serve live traffic.

Reference context

The project design was informed by the user-provided Uber DeeprETA reference paper. Its main applicable concept is a learned ETA adjustment layered on top of a routing-engine estimate, using rich feature interactions and continuous monitoring. This project extends that idea toward mountainous travel by emphasizing terrain, grade, weather, long routes, and uncertainty intervals.

Final conclusion

The project is now structured to become a credible mountain ETA system: it can ingest live routing and environmental context, preserve supervised labels, train ordered quantiles, evaluate them chronologically, and prevent an unproven checkpoint from taking control of serving. The next determinant of quality is disciplined collection of real, representative trip outcomes and measured comparison against the traffic-routing baseline.